

Part 3: Networking

Chapter 1: Why Can't Users Reach the Website?

It was Tuesday morning.

The engineering team had just finished migrating their application to the cloud when Maya's phone rang.

She answered immediately.

> "Hello?... Wait, users can't access the website?"

She hung up and looked at the team.

> "We've got a production incident."

Everyone rushed into the conference room.

Chris opened the monitoring dashboard.

Everything looked healthy.

CPU usage was normal.

Memory usage was low.

The Kubernetes Pods were running.

The database was online.

Alex frowned.

> "If all our services are healthy, why can't users reach the website?"

Maya smiled.

> "Because healthy servers don't always mean a healthy **network**."

Following the Request

Maya walked to the whiteboard.

She drew a simple diagram.

User's Laptop

↓

Home Router

↓

Internet

↓

Cloud Infrastructure

↓

****Application****

> "Before we fix anything, we need to understand how data travels."

Emma asked,

> "How does a message travel across the Internet?"

Chris replied,

> "It doesn't travel as one large file."

He drew several small rectangles.

> "Every message is divided into ****packets****."

Alex nodded.

> "Each packet travels independently until it reaches its destination."

Every Device Needs an Address

David asked,

> "How does a packet know where to go?"

Chris answered,

> "Every device connected to a network has an ****IP address****."

He pointed to his laptop.

> "Your computer has one."

Then he pointed to the cloud servers.

> "Our servers have one too."

Emma asked,

> "Is that like a home address?"

> "Exactly."

> "Without an IP address, other devices wouldn't know where to send data."

MAC Addresses

James noticed another address on the network configuration.

> "What's this long hexadecimal number?"

Chris smiled.

> "That's the **MAC address**."

Emma asked,

> "How is it different from an IP address?"

Chris explained,

> "An IP address identifies a device on a network."

> "A MAC address identifies the physical network interface."

Alex summarized.

> "IP addresses can change."

> "MAC addresses belong to the network hardware."

Chris nodded.

> "Exactly."

The Network Interface Card

David opened his laptop settings.

> "So where is the MAC address stored?"

Chris replied,

> "Inside the **Network Interface Card**, or **NIC**."

Emma smiled.

> "Every laptop, server, and virtual machine has one."

Local and Wide Networks

Maya drew two circles.

The first represented the office.

The second represented the Internet.

> "Inside this building we have a **Local Area Network (LAN)**."

She drew a much larger cloud around it.

> "The Internet is a **Wide Area Network (WAN)**."

Alex laughed.

> "So our office network is tiny compared to the Internet."

Switches

James pointed toward the server rack.

> "What's that device with all the Ethernet cables?"

Chris answered,

> "That's a **network switch**."

Emma asked,

> "What does it do?"

Chris replied,

> "A switch connects devices inside the same LAN."

He continued,

> "Instead of broadcasting data everywhere, it forwards packets only to the correct device."

Alex smiled.

> "That's much more efficient."

Routers

David asked another question.

> "Then how do packets leave our office?"

Chris pointed to another device.

> "Through the **router**."

He explained,

> "A router connects different networks together."

Emma nodded.

> "So our home router connects our LAN to the Internet."

> "Exactly."

The Default Gateway

Alex noticed another configuration.

> "My computer has something called a **default gateway**."

Chris explained,

> "Whenever your computer doesn't know where to send a packet, it sends it to the default gateway."

Emma laughed.

> "It's like asking someone for directions."

A Packet's Journey

Maya challenged the team.

> "Let's follow one packet."

She wrote:

****Laptop****

↓

****Switch****

↓

****Router****

↓

****Internet****

↓

****Cloud Router****

↓

****Load Balancer****

↓

****Application Server****

Alex smiled.

> "That's much longer than I expected."

Chris nodded.

> "A single packet may travel through dozens of routers before reaching its destination."

Testing Connectivity

Emma opened her terminal.

> "Let's see if our server is reachable."

She typed:

```
`ping ourapp.com`
```

The terminal displayed:

```
**Request timed out.**
```

David frowned.

```
> "The server isn't responding."
```

Chris wasn't worried.

```
> "Ping only tells us whether a device responds to ICMP requests."
```

Maya added,

```
> "A failed ping doesn't always mean the server is down."
```

The Problem

James checked the cloud dashboard.

Suddenly he smiled.

```
> "I found it."
```

Everyone looked at him.

```
> "Someone accidentally changed the route table."
```

Emma looked surprised.

```
> "So packets don't know where to go?"
```

```
> "Exactly."
```

Chris restored the correct route.

Emma tried again.

```
`ping ourapp.com`
```

```
**Reply received.**
```

Alex refreshed the browser.

The website loaded instantly.

Everyone applauded.

What They Learned

Before ending the meeting, Maya summarized the lesson.

She wrote the path on the whiteboard one final time.

****Application****

↑

****Server****

↑

****Router****

↑

****Internet****

↑

****Home Router****

↑

****Laptop****

> "Every request depends on dozens of networking components."

She smiled.

> "If just one component fails, users may never reach the application."

Alex closed his laptop.

> "Cloud engineering taught us where applications run."

Emma smiled.

> "Networking teaches us how users reach them."

The team left the conference room ready to dive even deeper into the world of computer networks.

Part 3: Networking

Chapter 2: The Mystery of the Slow Website

The next morning, the engineering team received another alert.

Unlike yesterday, users could reach the website.

However, many of them complained that pages loaded very slowly.

Maya gathered everyone in the conference room.

> "Yesterday we solved a connectivity problem."

She smiled.

> "Today we're solving a performance problem."

Alex opened the monitoring dashboard.

> "The servers aren't overloaded."

James checked the database.

> "Queries are fast."

Emma looked puzzled.

> "Then where's the delay?"

Chris walked to the whiteboard.

> "To answer that, we need to understand **TCP/IP**."

The TCP/IP Model

Chris drew four layers.

Application Layer

↓

Transport Layer

↓

Internet Layer

↓

Network Access Layer

Emma asked,

> "Do all network requests pass through these layers?"

> "Exactly," Chris replied.

> "Each layer has a specific responsibility."

Maya added,

> "The application doesn't need to know how packets travel."

> "Each layer handles its own job."

TCP vs. UDP

David asked,

> "Our application uses TCP."

> "Why not UDP?"

Chris drew two columns.

****TCP****

Reliable

Ordered

Connection-oriented

Error checking

****UDP****

Fast

Connectionless

No delivery guarantee

Alex nodded.

> "So TCP is slower but more reliable."

> "Exactly."

Emma added,

> "Video streaming or online games often prefer UDP because speed is more important than perfect reliability."

Establishing a Connection

Chris continued.

> "Before TCP sends any data, it establishes a connection."

He drew three arrows.

****Client → SYN → Server****

****Server → SYN-ACK → Client****

****Client → ACK → Server****

David smiled.

> "That's the famous ****three-way handshake****."

Chris nodded.

> "Only after the handshake does data begin to flow."

Ports

Emma examined the server configuration.

> "Our server has one IP address."

She looked confused.

> "How can multiple applications use the same address?"

Chris replied,

> "Because they listen on different **ports**."

He wrote a few examples.

- * HTTP → Port 80
- * HTTPS → Port 443
- * SSH → Port 22
- * PostgreSQL → Port 5432

Alex smiled.

> "So the IP identifies the machine."

> "The port identifies the application."

Sockets

James asked,

> "Where does a socket fit into this?"

Chris answered,

> "A **socket** combines an IP address and a port."

Emma nodded.

> "So every network connection is established between two sockets."

> "Exactly."

Firewalls

David tried connecting directly to the database.

The request failed immediately.

Chris smiled.

> "That's intentional."

He opened the cloud security settings.

> "Our **firewall** blocks database traffic from the public Internet."

Emma nodded.

> "Only the backend application is allowed to connect."

Maya added,

> "A firewall filters incoming and outgoing traffic according to security rules."

Packet Loss

Alex ran another network test.

Several packets disappeared.

> "We're seeing **packet loss**."

Emma frowned.

> "What happens when TCP loses a packet?"

Chris replied,

> "TCP automatically retransmits it."

James smiled.

> "Reliability comes at the cost of extra time."

Retransmission

Chris opened a packet capture.

Everyone watched as one packet disappeared.

A few milliseconds later, the sender transmitted it again.

Emma pointed at the screen.

> "There's the **retransmission**."

Chris nodded.

> "TCP assumes the packet was lost if it doesn't receive an acknowledgment."

Bandwidth and Throughput

David noticed another graph.

> "Bandwidth usage is low."

Emma asked,

> "Is bandwidth the same as throughput?"

Chris shook his head.

> "Not exactly."

He explained,

> **"Bandwidth"** is the maximum amount of data the network can carry."

> **"Throughput"** is the amount of data that actually travels."

Alex smiled.

> "So traffic jams reduce throughput even if bandwidth is high."

Latency

James pointed to another graph.

> "Latency has increased."

Maya explained,

> "Latency measures how long data takes to travel."

Emma added,

> "Even a fast server feels slow if the network latency is high."

Chris nodded.

> "Exactly."

Connection Timeout

While testing the payment service, Alex received an error.

Connection Timeout

David asked,

> "What causes that?"

Chris replied,

> "The client waited too long for the server to respond."

Emma added,

> "Timeouts often happen because of network congestion, firewall rules, or overloaded services."

Half-Open Connections

James checked the load balancer.

> "We have thousands of half-open connections."

Alex looked confused.

Chris explained,

> "Some clients started the TCP handshake but never completed it."

Emma asked,

> "Is that normal?"

> "Sometimes."

Maya added,

> "But attackers also exploit this behavior in ****SYN flood attacks****."

The team made a note to discuss network security in a later chapter.

Solving the Mystery

Chris continued investigating.

Finally, he found the issue.

> "Our firewall is inspecting every packet with an outdated rule set."

Emma asked,

> "Can we optimize it?"

Chris nodded.

After simplifying the rules, latency immediately dropped.

Packet retransmissions disappeared.

The website became responsive again.

Alex refreshed the dashboard.

Everything was green.

Wrapping Up

Before leaving the room, Maya summarized the day's lessons.

She wrote several terms on the whiteboard.

- * TCP
- * UDP
- * Three-Way Handshake
- * Port
- * Socket
- * Firewall
- * Packet Loss
- * Retransmission
- * Bandwidth
- * Throughput
- * Latency
- * Timeout

She smiled.

> "Yesterday we learned how packets travel."

She looked around the room.

> "Today we learned how reliable communication happens."

Emma closed her laptop.

> "Networking isn't just about connecting computers."

Alex smiled.

> "It's about making communication fast, reliable, and secure."

The team left the office better prepared for the next challenge, where they would discover what actually happens when a web browser sends an HTTP request to a server.

Part 3: Networking

Chapter 3: What Happens When You Click "Login"?

Thursday morning began with an interesting question.

As everyone settled into the conference room, Maya projected the company's login page onto the screen.

She looked around the room.

> "Every day, millions of people click this **Login** button."

She smiled.

> "But what actually happens after that single click?"

Alex laughed.

> "A lot more than most people imagine."

The Browser Sends a Request

Emma opened the browser's **Developer Tools**.

She clicked the **Network** tab.

> "Let's watch every request."

She entered her email and password before pressing **Login**.

Immediately, a new entry appeared.

POST /login

David pointed at the screen.

> "That's an **HTTP request**."

Chris nodded.

> "The browser acts as the **client**, and our application is the **server**."

HTTP Methods

Maya wrote several words on the whiteboard.

- * GET
- * POST
- * PUT
- * PATCH
- * DELETE

Alex asked,

> "Why are there different methods?"

Chris explained.

> **GET** retrieves data."

> **POST** creates a new resource."

> **PUT** replaces an existing resource."

> **PATCH** updates part of a resource."

> **DELETE** removes a resource."

Emma smiled.

> "So logging in uses POST because we're sending credentials."

Request Headers

David clicked the request.

Dozens of lines appeared.

Emma asked,

> "What are all these?"

Chris replied,

> "They're ****HTTP headers****."

He highlighted several.

****Host****

****User-Agent****

****Content-Type****

****Authorization****

****Accept****

Maya explained,

> "Headers contain metadata about the request."

Alex nodded.

> "They're like information written on the outside of a package."

The Request Body

James pointed lower on the screen.

> "Here's the email and password."

Chris smiled.

> "That's the ****request body****."

Emma added,

> "GET requests usually don't have one."

> "POST requests often do."

HTTPS Protects the Request

David asked,

> "Can someone read my password while it's traveling across the Internet?"

Chris shook his head.

> "No."

He pointed to the padlock icon.

> "The browser uses ****HTTPS****, which encrypts communication using ****TLS****."

Maya added,

> "Anyone intercepting the traffic would only see encrypted data."

Everyone felt reassured.

Reaching the Server

The request arrived at the backend.

Alex opened the application logs.

> "The authentication service received the request."

Emma smiled.

> "Now the server validates the user's credentials."

James checked the database.

The email existed.

The password hash matched.

> "Authentication successful."

Creating a Session

David asked,

> "How does the server remember me after I log in?"

Chris explained,

> "Traditionally, the server creates a ****session****."

He drew a small table.

Session ID → User

Emma nodded.

> "The session stays on the server."

> "Exactly."

Cookies

Chris continued.

> "The server sends the browser a ****cookie****."

Alex looked at the response.

A ****Set-Cookie**** header appeared.

Emma smiled.

> "The browser stores it automatically."

Chris nodded.

> "Every future request includes that cookie."

David laughed.

> "So the server recognizes me without asking for my password every time."

JSON Web Tokens

Maya introduced another approach.

> "Many modern APIs use ****JSON Web Tokens (JWTs)**** instead of server-side sessions."

Emma asked,

> "How does that work?"

Chris explained.

> "After login, the server creates a signed token."

He continued.

> "The client stores it and sends it in the ****Authorization**** header."

Alex smiled.

> "The server verifies the signature without storing session data."

REST APIs

The frontend requested the user's profile.

Emma noticed another URL.

****GET /api/profile****

Chris smiled.

> "That's part of our **REST API**."

Maya explained,

> "REST organizes application functionality into resources."

David added,

> "Each endpoint performs a specific operation."

HTTP Status Codes

Suddenly, one request failed.

The browser displayed:

404 Not Found

Alex laughed.

> "Wrong URL."

Chris wrote several status codes on the whiteboard.

```
* **200 OK**
* **201 Created**
* **204 No Content**
* **301 Moved Permanently**
* **400 Bad Request**
* **401 Unauthorized**
* **403 Forbidden**
* **404 Not Found**
* **500 Internal Server Error**
* **503 Service Unavailable**
```

Emma smiled.

> "Those numbers tell clients exactly what happened."

Caching Responses

James noticed another response header.

Cache-Control

He asked,

> "What's this for?"

Chris replied,

> "It tells browsers whether a response can be **cached**."

Emma nodded.

> "Static images can be cached."

Alex added,

> "User account information shouldn't be."

Content Negotiation

David inspected another header.

****Accept: application/json****

Chris explained,

> "The client tells the server which response format it prefers."

Emma smiled.

> "The same endpoint could return JSON, XML, or another supported format."

Keep-Alive Connections

Alex noticed something interesting.

> "The browser isn't opening a new TCP connection for every request."

Chris nodded.

> "That's because of ****HTTP Keep-Alive****."

He explained,

> "Reusing connections reduces latency and improves performance."

A Small Bug

During testing, Emma couldn't log in.

The browser displayed:

****401 Unauthorized****

James checked the logs.

> "The password is correct."

Chris inspected the request.

A few seconds later, he smiled.

> "The frontend forgot to include the Authorization header after refreshing the token."

Emma fixed the issue.

The next request returned:

****200 OK****

Everyone applauded.

Following the Complete Journey

Before ending the meeting, Maya summarized the entire process.

She wrote one long sequence on the whiteboard.

****User Clicks Login****

↓

Browser Creates an HTTP POST Request

↓

HTTPS Encrypts the Traffic

↓

Request Travels Across the Internet

↓

Load Balancer

↓

Backend API

↓

Authentication Service

↓

Database Verification

↓

JWT Token Generated

↓

HTTP Response (200 OK)

↓

Browser Stores the Token

↓

User Successfully Logged In

She smiled.

> "One click."

Alex laughed.

> "Hundreds of operations."

Emma closed her laptop.

> "Understanding HTTP makes web applications feel much less mysterious."

The team left the meeting knowing that every successful login was the result of dozens of networking technologies working together seamlessly.

Part 3: Networking

Chapter 4: Where Does `ourapp.com` Really Go?

Friday morning, Maya walked into the conference room carrying a simple question.

She wrote a URL on the whiteboard.

https://ourapp.com

Then she turned to the team.

> "Yesterday we learned what happens after the browser sends an HTTP request."

She paused.

> "But before the browser can send that request, it has to answer one important question."

Emma smiled.

> "Where is **ourapp.com**?"

Maya nodded.

> "Exactly."

A Browser Doesn't Understand Domain Names

Alex typed `**https://ourapp.com**` into his browser.

Nothing happened for a split second.

David asked,

> "What's the browser doing?"

Chris answered,

> "The browser doesn't know the server's IP address."

He pointed to the screen.

> "First, it performs a `**DNS lookup**`."

Emma nodded.

> "So DNS translates domain names into IP addresses."

> "Exactly."

Checking the Browser Cache

Chris asked,

> "Where should the browser look first?"

James replied,

> "Its own `**DNS cache**`."

Chris smiled.

> "Correct."

If the browser had recently visited the website, it could reuse the cached result without asking anyone else.

Emma added,

> "That saves time."

The Operating System Cache

The browser cache was empty.

Chris continued.

> "Next, the browser asks the operating system."

Alex nodded.

> "The operating system also keeps a DNS cache."

Again, no record existed.

The search continued.

The Recursive Resolver

Chris drew another box.

****Recursive DNS Resolver****

David asked,

> "Who operates that?"

> "Usually your Internet Service Provider," Chris replied.

> "Or a public DNS service like Google Public DNS or Cloudflare DNS."

Emma smiled.

> "So the resolver does all the hard work for us."

Root Name Servers

The recursive resolver didn't know the answer either.

Chris drew another arrow.

> "Now it contacts a ****Root Name Server****."

Alex asked,

> "Does the root server know every website?"

Chris laughed.

> "No."

> "It simply knows where to find the next level."

Top-Level Domain Servers

The root server replied.

****Ask the .com servers.****

Chris continued.

> "Those are called ****Top-Level Domain (TLD) Name Servers****."

Emma nodded.

> "So every `.com` domain eventually reaches a `.com` name server."

The Authoritative Name Server

The TLD server replied with another address.

Chris pointed to the final box.

> "Now the resolver contacts the **Authoritative Name Server**."

James smiled.

> "Finally."

Chris nodded.

> "This server owns the DNS records for our domain."

It responded with:

ourapp.com → 34.210.18.25

Emma smiled.

> "Now the browser knows where to send the HTTP request."

Understanding DNS Records

Maya opened the DNS management console.

Several records appeared.

Alex asked,

> "Why are there so many record types?"

Chris explained them one by one.

A Record

> "Maps a domain name to an IPv4 address."

AAAA Record

> "Maps a domain name to an IPv6 address."

CNAME Record

> "Creates an alias for another domain."

MX Record

> "Specifies which mail servers receive emails."

TXT Record

> "Stores additional information, often used for domain verification and email security."

****NS Record****

> "Identifies the authoritative name servers for the domain."

Emma nodded.

> "Each record has a different purpose."

Time To Live (TTL)

David noticed another field.

****TTL = 300 seconds****

He asked,

> "What's TTL?"

Chris replied,

> ****Time To Live**** tells DNS resolvers how long they may cache a record."

Emma smiled.

> "A longer TTL reduces DNS traffic."

Alex added,

> "But changes take longer to appear."

> "Exactly."

DNS Propagation

The team updated the application's IP address.

Emma refreshed the browser.

Nothing changed.

She frowned.

> "Did the update fail?"

Chris shook his head.

> "No."

> "You're seeing ****DNS propagation****."

He explained,

> "Resolvers around the world update their caches gradually."

James smiled.

> "So different users may receive different answers for a while."

Diagnosing DNS Problems

Alex opened his terminal.

> "Let's verify the record."

He typed:

```
`nslookup ourapp.com`
```

The correct IP address appeared.

Emma used another command.

```
`dig ourapp.com`
```

The response showed:

```
* Resolver
* TTL
* A Record
* Authoritative Server
```

Chris smiled.

> "These tools are essential for DNS troubleshooting."

Reverse DNS

David asked another question.

> "Can DNS work in reverse?"

Chris nodded.

> "Yes."

> "***Reverse DNS** translates an IP address back into a domain name."

Emma laughed.

> "So DNS works in both directions."

A Real Problem

Just before lunch, QA reported another issue.

European users couldn't access the website.

Everyone checked the servers.

Everything was healthy.

Chris inspected the DNS records.

A few moments later, he smiled.

> "I found the problem."

Emma asked,

> "What is it?"

> "The European DNS zone is missing an A Record."

Alex added the missing record.

After a few minutes, users across Europe could access the application again.

DNS Load Balancing

James noticed another feature.

> "Our domain has two A Records."

Chris nodded.

> "That's intentional."

He explained,

> "DNS can return different IP addresses."

Emma smiled.

> "So traffic is distributed between multiple servers."

> "Exactly."

Wrapping Up

Before ending the meeting, Maya summarized the complete DNS journey.

She drew one final diagram.

****Browser****

↓

Browser DNS Cache

↓

Operating System Cache

↓

Recursive Resolver

↓

Root Name Server

↓

TLD Name Server

↓

Authoritative Name Server

↓

DNS Record Returned

↓

Browser Connects to the Server

She looked around the room.

> "Without DNS, users would have to remember IP addresses."

Alex laughed.

> "I definitely wouldn't remember `34.210.18.25`."

Emma smiled.

> "Fortunately, I only need to remember `ourapp.com`."

Everyone laughed.

As the meeting ended, the team realized that every website visit begins with a remarkably sophisticated conversation between computers—a conversation most users never notice.

Part 3: Networking

Chapter 5: Hunting the Invisible Problem

Monday morning was peaceful.

The monitoring dashboards were green.

The APIs were responding normally.

CPU usage was low.

Memory usage looked healthy.

Yet the customer support team kept receiving complaints.

> "Sometimes the website loads instantly."

> "Sometimes it takes twenty seconds."

> "Sometimes it doesn't load at all."

Emma looked confused.

> "The problem is **intermittent**."

Maya nodded.

> "Those are often the hardest problems to solve."

She looked around the room.

> "Today we're going to think like network engineers."

Starting with Ping

Chris opened his terminal.

> "We'll begin with the simplest tool."

He typed:

```
```bash
ping ourapp.com
```
```

The terminal replied:

```
```
Reply from 34.210.18.25
Time = 18 ms
```
```

Alex smiled.

> "Looks healthy."

Chris continued running the command.

Most replies arrived within twenty milliseconds.

Then suddenly...

```
```
Request timed out.
```
```

Emma frowned.

> "So connectivity comes and goes."

Maya nodded.

> "Exactly."

Following the Route

Chris opened another terminal.

```
```bash
traceroute ourapp.com
```
```

David watched the output.

Each line represented another ****hop****.

Emma asked,

> "What's a hop?"

Chris explained,

> "Every time a packet passes through a router, that's one hop."

Alex counted them.

> "Our packets cross twelve routers before reaching AWS."

Where Is the Delay?

The first few hops responded quickly.

Hop 1

1 ms

Hop 2

3 ms

Hop 3

5 ms

Everything looked normal.

Then...

Hop 8

850 ms

Emma pointed at the screen.

> "That's our delay."

Chris nodded.

> "Somewhere after Hop 7, packets begin slowing down."

Using MTR

James suggested another tool.

> "Let's use **MTR**."

Alex asked,

> "How is it different from traceroute?"

Chris replied,

> "Traceroute gives us one snapshot."

> "**MTR (My Traceroute)** continuously measures latency and packet loss."

After several minutes, MTR showed that Hop 8 occasionally dropped packets.

Emma smiled.

> "Now we have consistent evidence."

Testing the API

David wanted to isolate the application.

He typed:

```
```bash
curl https://ourapp.com/api/health
```
```

The server replied:

```
```json
{
 "status": "healthy"
}
```
```

Alex nodded.

> "The API works."

Chris smiled.

> "Curl is perfect for testing HTTP endpoints without using a browser."

Capturing Packets

Maya wanted deeper visibility.

Chris opened another terminal.

```
```bash
tcpdump
```
```

Packets immediately began scrolling across the screen.

Emma looked overwhelmed.

> "That's a lot of information."

Chris laughed.

> "We're performing a **packet capture**."

He continued,

> "Every packet entering or leaving this server can be inspected."

Wireshark

James preferred a graphical interface.

He opened **Wireshark**.

Colorful packets filled the screen.

Alex asked,

> "Why use Wireshark instead of tcpdump?"

Chris replied,

> "Tcpdump is excellent for servers."

> "Wireshark makes packet analysis much easier."

Emma followed one TCP conversation from start to finish.

> "Now I can actually see the handshake."

ARP

David noticed another protocol.

> "What's **ARP**?"

Chris explained,

> "***Address Resolution Protocol** translates an IP address into a MAC address inside a local network."

Emma smiled.

> "So DNS translates domain names into IP addresses."

> "ARP translates IP addresses into hardware addresses."

> "Exactly."

MTU Problems

James discovered something unusual.

Large requests often failed.

Small ones worked perfectly.

Chris immediately suspected one thing.

> "***MTU**."

Emma asked,

> "Maximum Transmission Unit?"

Chris nodded.

> "It's the largest packet size a network can carry."

Fragmentation

Alex tested a larger payload.

The packet was split into smaller pieces.

Chris explained,

> "That's called **fragmentation**."

Emma frowned.

> "Does fragmentation hurt performance?"

> "It can."

> "Extra work is required to split and reassemble packets."

Routing Loop

While reviewing the routing tables, James noticed something alarming.

Two routers were forwarding packets to each other repeatedly.

David looked surprised.

> "They're stuck."

Chris nodded.

> "That's a ****routing loop****."

Emma watched the packet counter increase rapidly.

> "The packets never reach their destination."

Quality of Service

After fixing the routing loop, Maya introduced another topic.

> "Suppose voice calls and software updates use the same network."

Alex smiled.

> "Voice calls should have higher priority."

Chris nodded.

> "That's exactly what ****Quality of Service (QoS)**** does."

He explained,

> "QoS prioritizes important traffic when bandwidth is limited."

Emma added,

> "Video conferences should be smoother even during heavy downloads."

Measuring Jitter

James examined another graph.

> "Latency isn't constant."

Chris replied,

> "That's called ****jitter****."

Emma asked,

> "Why does it matter?"

Chris answered,

> "Video calls and online gaming are very sensitive to jitter."

Solving the Mystery

After hours of investigation, Chris finally found the root cause.

A recently installed router had the wrong MTU configuration.

Large packets were fragmented repeatedly.

Some fragments were dropped.

Others entered a routing loop before timing out.

The team corrected the MTU settings.

Immediately:

- * Ping became stable.
- * Traceroute showed normal response times.
- * MTR reported zero packet loss.
- * Curl returned responses instantly.
- * Wireshark confirmed that packets were no longer fragmented unnecessarily.

Emma smiled.

> "Everything is back to normal."

The Lesson

Before ending the day, Maya summarized everything they had used.

She wrote a checklist on the whiteboard.

- ☒ Ping
- ☒ Traceroute
- ☒ MTR
- ☒ Curl
- ☒ Tcpdump
- ☒ Wireshark
- ☒ Packet Capture
- ☒ ARP
- ☒ MTU
- ☒ Fragmentation
- ☒ Routing Loop
- ☒ QoS

✅ Jitter

She looked around the room.

> "Great engineers don't guess."

She smiled.

> "They investigate."

Alex packed up his laptop.

> "Every tool gave us one piece of the puzzle."

Emma nodded.

> "And together, they revealed the complete picture."

The team left the office with a new appreciation for the art of network troubleshooting, knowing that even the most invisible problems leave clues for those who know where to look.

Part 3: Networking

Chapter 6: Preparing for Millions of Users

A month had passed since the team solved the mysterious network issues.

The application was now stable.

Customers were happy.

Traffic continued to grow every day.

Then, during the weekly planning meeting, Maya shared exciting news.

> "Our marketing team is launching a global campaign next month."

She paused.

> "We're expecting nearly ten times more users."

The room became quiet.

Alex smiled.

> "So today's challenge isn't fixing the network."

> "It's preparing it for massive scale."

The Reverse Proxy

Chris updated the architecture diagram.

A new component appeared in front of the application servers.

Emma asked,

> "What's that?"

Chris replied,

> "It's a **reverse proxy**."

David looked curious.

> "What does it do?"

Chris explained,

> "Users send requests to the reverse proxy instead of directly to our application."

> "The reverse proxy forwards those requests to the appropriate backend service."

Maya added,

> "It can also handle SSL termination, caching, compression, and load balancing."

Alex smiled.

> "So it protects our backend while improving performance."

Forward Proxy

James asked,

> "Is that the same as a proxy server?"

Chris shook his head.

> "Not exactly."

He drew two diagrams.

> **Forward Proxy**

Client → Proxy → Internet

> **Reverse Proxy**

Client → Reverse Proxy → Server

Emma nodded.

> "A forward proxy represents the client."

> "A reverse proxy represents the server."

Network Address Translation (NAT)

David remembered another networking term.

> "How do hundreds of employees browse the Internet using one public IP address?"

Chris smiled.

> "That's **Network Address Translation (NAT)**."

He drew an office network.

Every computer had a private IP address.

Only the router had a public IP address.

Emma understood immediately.

> "The router translates private addresses into one public address."

> "Exactly."

Virtual Private Network

Several developers were working remotely.

Alex asked,

> "How do they securely access our internal systems?"

Chris replied,

> "Through a **Virtual Private Network (VPN)**."

Emma added,

> "So their laptops behave as if they're connected to the company network."

Chris nodded.

> "All traffic is encrypted before it travels across the Internet."

WebSockets

The product team announced another feature.

Real-time notifications.

David asked,

> "Should the browser keep sending HTTP requests every second?"

Chris smiled.

> "That would waste resources."

He drew one long arrow between the browser and the server.

> "Instead, we'll use **WebSockets**."

Emma asked,

> "So one connection stays open?"

> "Exactly."

> "The server can push updates immediately without waiting for another request."

Alex smiled.

> "Perfect for chat applications, stock prices, and live dashboards."

HTTP/2

Chris opened another presentation.

> "Our application still uses HTTP/1.1."

Emma asked,

> "What's better about **HTTP/2**?"

Chris explained,

> "HTTP/2 supports **multiplexing**."

Alex looked interested.

> "Meaning multiple requests share the same connection?"

> "Exactly."

David added,

> "That reduces latency and improves page loading."

HTTP/3 and QUIC

James noticed another protocol.

> "HTTP/3?"

Chris nodded.

> "HTTP/3 runs on **QUIC** instead of TCP."

Emma asked,

> "Why is that important?"

Chris explained,

> "QUIC uses UDP underneath while providing reliable communication."

Alex smiled.

> "So users experience faster connection establishment and better performance on unstable networks."

CDN Edge Computing

The marketing campaign attracted users from around the world.

Emma noticed that users in South America experienced slower downloads.

Chris suggested another improvement.

> "We'll expand our **Content Delivery Network**."

He pointed to several new **edge locations**.

> "Static files will now be served from servers closer to users."

James added,

> "Some application logic can even run at the edge."

Emma smiled.

> "That's **edge computing**."

Service Mesh

As more microservices were added, the architecture became increasingly complex.

Alex asked,

> "Managing communication between dozens of services is getting difficult."

Chris nodded.

> "That's why many organizations adopt a **service mesh**."

Emma asked,

> "What does it provide?"

Chris replied,

> "Traffic management, service discovery, encryption, retries, load balancing, and observability between services."

David smiled.

> "Without changing application code?"

> "Exactly."

Zero-Trust Networking

Sarah from the security team joined the discussion.

She wrote a sentence on the whiteboard.

****Never Trust. Always Verify.****

Emma smiled.

> "That's ****Zero Trust****."

Sarah nodded.

> "Every request must be authenticated and authorized."

Alex added,

> "Even if it comes from inside our own network."

> "Exactly."

Chris explained,

> "Modern networks assume that no connection is automatically trustworthy."

Preparing for Launch

The team reviewed the final architecture.

Browser

↓

CDN

↓

Reverse Proxy

↓

Load Balancer

↓

Kubernetes Cluster

↓

Service Mesh

↓

Microservices

↓

Database

↓

Monitoring Platform

Emma smiled.

> "That's a long journey."

Maya nodded.

> "But every component has a purpose."

Launch Day

One week later, the global campaign began.

Traffic doubled.

Then tripled.

Then increased tenfold.

The dashboards filled with activity.

The CDN served millions of images.

The reverse proxy distributed incoming requests.

The load balancer spread traffic evenly.

WebSockets delivered live notifications instantly.

HTTP/2 reused connections efficiently.

HTTP/3 improved performance for mobile users.

The service mesh handled communication between dozens of services.

Zero Trust policies protected every request.

The system remained responsive.

Customers never noticed how much work happened behind the scenes.

Final Review

That evening, Maya gathered the team one last time.

She smiled proudly.

> "When we started this journey, we asked a simple question."

She pointed toward the login page projected on the wall.

> "How does one user reach our application?"

She looked around the room.

> "Now we know the answer."

Alex laughed.

> "It's not one technology."

Emma smiled.

> "It's hundreds of technologies working together."

Chris closed his laptop.

> "Networking isn't just about cables and routers."

James nodded.

> "It's about connecting people to applications quickly, securely, and reliably."

The team celebrated another successful release, knowing that behind every click, every message, and every webpage was an invisible network engineered to keep the digital world connected.